



DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale

Jialiang Huang^{1,2}, Hongxuan Tang¹, Jingchang Chen¹, Yuxuan Liu¹, Yixiao Chen¹, et al.
¹ DeepSeek-AI ² Tsinghua University

PREPRINT 2026

Problem

Agentic training creates **bursty, stateful** sandboxes; persistent memory, huge image corpora, and mixed isolation exceed any single runtime.

SCALE GAP

One job may request up to **32,000** stateful sandboxes.

Motivation

- Eager pulls and monolithic environments delay creation bursts.
- Uncontrolled overcommit duplicates memory and harms latency-sensitive agent steps.

EAGER +
MONOLITHIC

Pull and unpack every image before work.

vs.

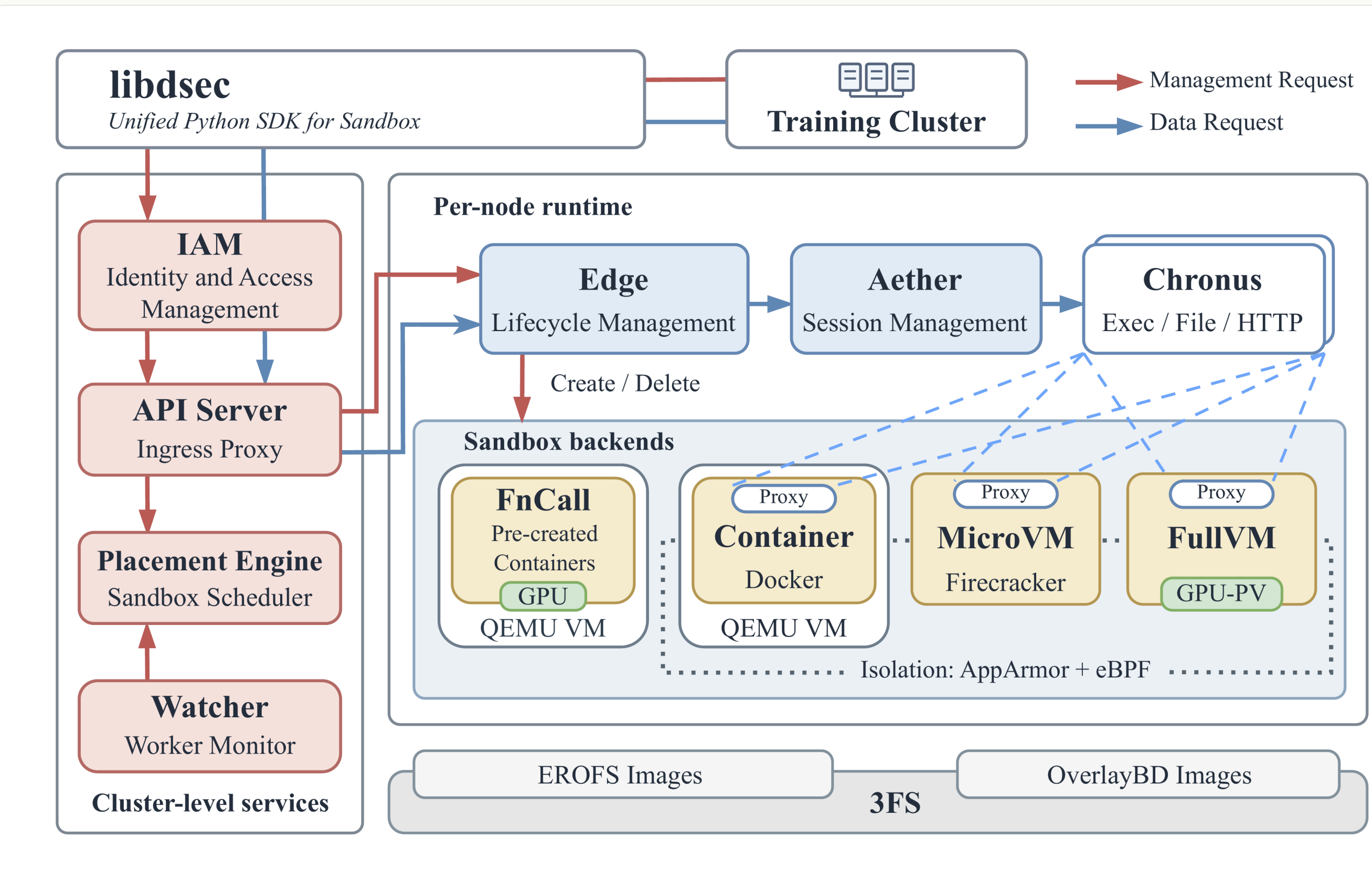
DSEC

Compose layers; fetch only the working set.

CPU demand is intermittent, but memory and accumulated state persist between agent actions.

Low reuse and long sessions make elasticity a system-wide requirement.

Method



DSec architecture: one SDK, four sandbox backends, and 3FS-backed images.

A Runtime Portfolio

match isolation to work

- FnCall** serves short, pre-created execution.
- Containers** cover broad Linux workloads efficiently.
- MicroVMs and full VMs** add stronger isolation and compatibility.

FnCall Container MicroVM Full VM

B Elastic Control

one request path

SDK → API → Place → Edge → Run

The control plane separates global placement from node-local lifecycle management, allowing large bursts to scale horizontally.

Global scheduling, local lifecycle control, one unified SDK.

C Composable Environments

load only what runs

Immutable **EROFS** layers keep metadata local while **3FS** serves file data lazily, avoiding full-image download and repeated extraction.

1 Compose > 2 Mount > 3 Fetch on demand

D Dense Execution

share • reclaim • protect

- virtio-pmem shares read-only mappings across microVMs.
- DAMON plus balloon FPR reclaims idle guest memory at scale.
- SCHED_IDLE plus core scheduling protects latency-sensitive work.

40.2%

lower peak memory

21.2%

lower integrated memory

17.3%

latency inflation at 50% load

MECHANISM

PRESSURE CONTROLLED

MEASURED OUTCOME

virtio-pmem

Duplicate page cache

–40.2% peak memory

DAMON + FPR

Idle guest pages

–21.2% integrated memory

SCHED_IDLE + core

SMT interference

17.3% vs 45.2%

These controls are complementary: sharing removes duplicate read-only state, reclamation recovers idle pages, and CPU QoS protects interactive steps.

Key Results

IMAGE PATH	COMPLETION	WRITES / NODE
Docker • cached	≈35 min	≈600 GB
Docker • eager	>60 min	>1,600 GB
DSec • EROFS	≈35 min	≈700 GB

≈57% fewer disk writes than eager image pulling

SO WHAT → Lazy data access approaches fully-local speed while avoiding eager distribution and extraction.

Headline Numbers

380K+
concurrent sandboxes
production peak

5K+/s sandbox creations

3M/day sandboxes served

≈160 nodes per unit

Takeaway

DSec scales agentic RL by co-designing isolation, data access, lifecycle, memory, and CPU control while preserving rollout state across preemption.

Production proof: 380K+ concurrent sandboxes.

Scan to Read



Paper